

Aaditya Dawar

1024150093

2013

Assignment-8: Function and Class Templates

Function Template

1. Write a function template to swap two variables of any data type.

Code-

```
#include <iostream>
```

```
using namespace std;
```

```
// Function template
```

```
template <typename T>
```

```
void swapValues(T &a, T &b) {
```

```
    T temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
int main() {
```

```
    int x, y;
```

```
    cout << "Enter two integers: ";
```

```
    cin >> x >> y;
```

```
    cout << "Before swap: x = " << x << ", y = " << y << endl;
```

```
    swapValues(x, y);
```

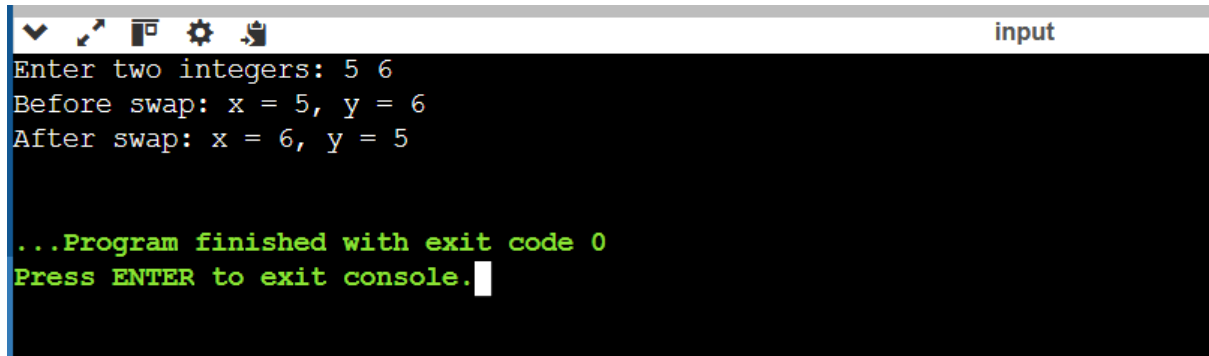
```

    cout << "After swap: x = " << x << ", y = " << y << endl;

    return 0;
}

```

OUTPUT-



The screenshot shows a terminal window with a dark background and a light gray title bar labeled 'input'. The terminal displays the following text: 'Enter two integers: 5 6', 'Before swap: x = 5, y = 6', and 'After swap: x = 6, y = 5'. Below this, it shows the program's completion: '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor at the end of the line.

```

Enter two integers: 5 6
Before swap: x = 5, y = 6
After swap: x = 6, y = 5

...Program finished with exit code 0
Press ENTER to exit console.

```

2. Write a function template to find the minimum element in an array.

Code- #include <iostream>

using namespace std;

// Function template to find minimum element

template <typename T>

T findMin(T arr[], int n) {

 T min = arr[0];

 for (int i = 1; i < n; i++) {

 if (arr[i] < min) {

 min = arr[i];

 }

 }

 return min;

}

```

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

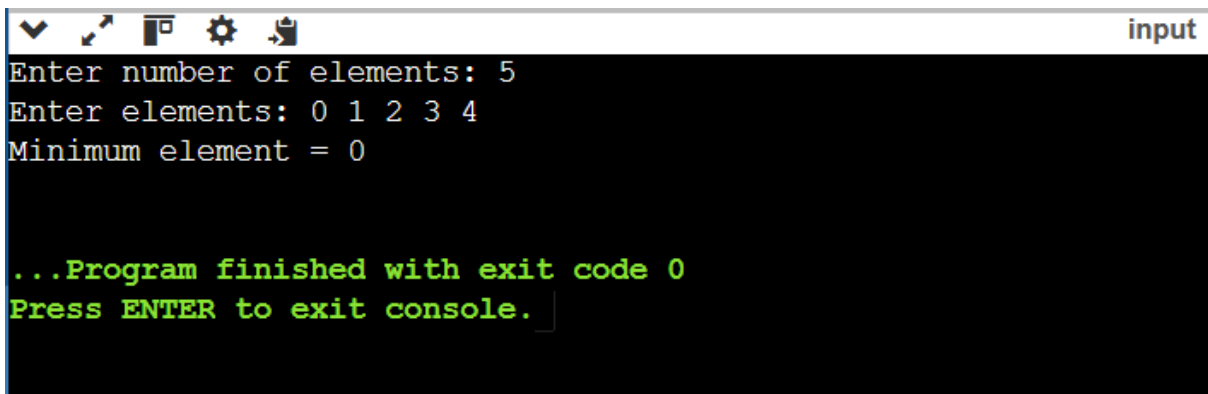
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Minimum element = " << findMin(arr, n) << endl;

    return 0;
}

```

OUTPUT-



```

input
Enter number of elements: 5
Enter elements: 0 1 2 3 4
Minimum element = 0

...Program finished with exit code 0
Press ENTER to exit console.

```

3. Write a function template to sort an array using bubble sort.

Code-

```

#include <iostream>

using namespace std;

```

```

// Function template for Bubble Sort
template <typename T>
void bubbleSort(T arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    bubbleSort(arr, n);
}

```

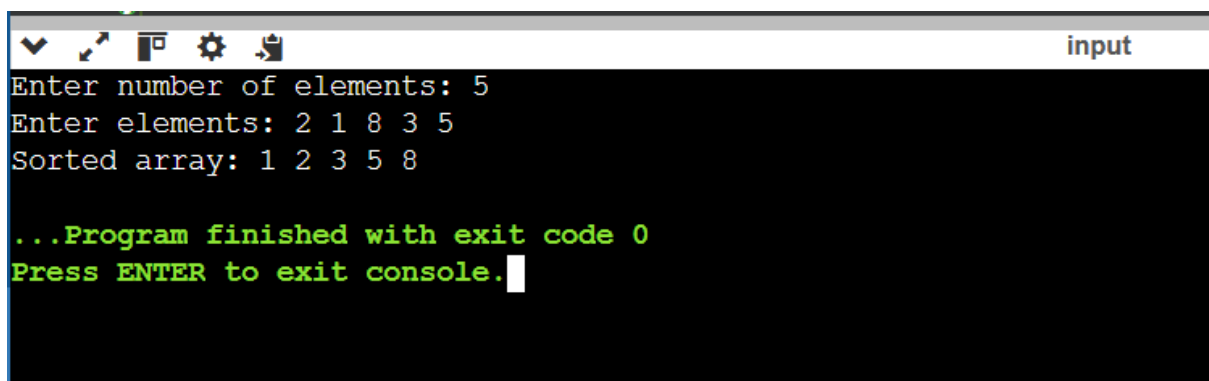
```

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}

```

OUTPUT-



```

input
Enter number of elements: 5
Enter elements: 2 1 8 3 5
Sorted array: 1 2 3 5 8

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Write a function template to perform linear search.

Code-

```

#include <iostream>

using namespace std;

template <typename T>
int linearSearch(T arr[], int n, T key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

```

```
int main() {  
    int n;  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    int key;  
    cout << "Enter element to search: ";  
    cin >> key;  
  
    int result = linearSearch(arr, n, key);  
  
    if (result != -1)  
        cout << "Element found at index: " << result << endl;  
    else  
        cout << "Element not found!" << endl;  
  
    return 0;  
}
```

OUTPUT-

```
input
Enter number of elements: 5
Enter elements: 5 4 8 0 2
Enter element to search: 8
Element found at index: 2

...Program finished with exit code 0
Press ENTER to exit console.□
```

5. Design overloaded versions of a function template process() such that:

- i) Accepts a single parameter
- ii) Accepts two parameters of the same type
- iii) Accepts two parameters of different types

Code-

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T>
```

```
void process(T a) {
```

```
    cout << "Single parameter: " << a << endl;
```

```
}
```

```
template <typename T>
```

```
void process(T a, T b) {
```

```
    cout << "Two parameters (same type): " << a << ", " << b << endl;
```

```
}
```

```
template <typename T, typename U>
```

```
void process(T a, U b) {
```

```
    cout << "Two parameters (different types): " << a << ", " << b << endl;
```

```
}
```

```
int main() {
```

```
int a, b;

double c;

cout << "Enter one integer: ";

cin >> a;

process(a);

cout << "Enter two integers: ";

cin >> a >> b;

process(a, b);

cout << "Enter an integer and a double: ";

cin >> a >> c;

process(a, c);

return 0;

}
```

Output-

```
Enter one integer: 5
Single parameter: 5
Enter two integers: 2 5
Two parameters (same type): 2, 5
Enter an integer and a double: 2 55
Two parameters (different types): 2, 55

...Program finished with exit code 0
Press ENTER to exit console.□
```

Class Template

1. Develop a class template for stack with push and pop operations.

```
#include <iostream>

using namespace std;

template <class T>

class Stack {

    T arr[100];

    int top;

public:

    Stack() { top = -1; }

    void push(T x) {

        if(top == 99) {

            cout << "Stack Overflow\n";

            return;

        }

        arr[++top] = x;

    }

    void pop() {

        if(top == -1) {

            cout << "Stack Underflow\n";

            return;

        }

        cout << "Popped: " << arr[top--] << endl;

    }

    void display() {
```

```

        if(top == -1) {
            cout << "Stack is empty\n";
            return;
        }
        for(int i = top; i >= 0; i--) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

```

```

int main() {
    Stack<int> s;
    int choice, val;

    do {
        cout << "1.Push 2.Pop 3.Display 4.Exit\n";
        cin >> choice;

        switch(choice) {
            case 1:
                cin >> val;
                s.push(val);
                break;
            case 2:
                s.pop();
                break;
            case 3:
                s.display();

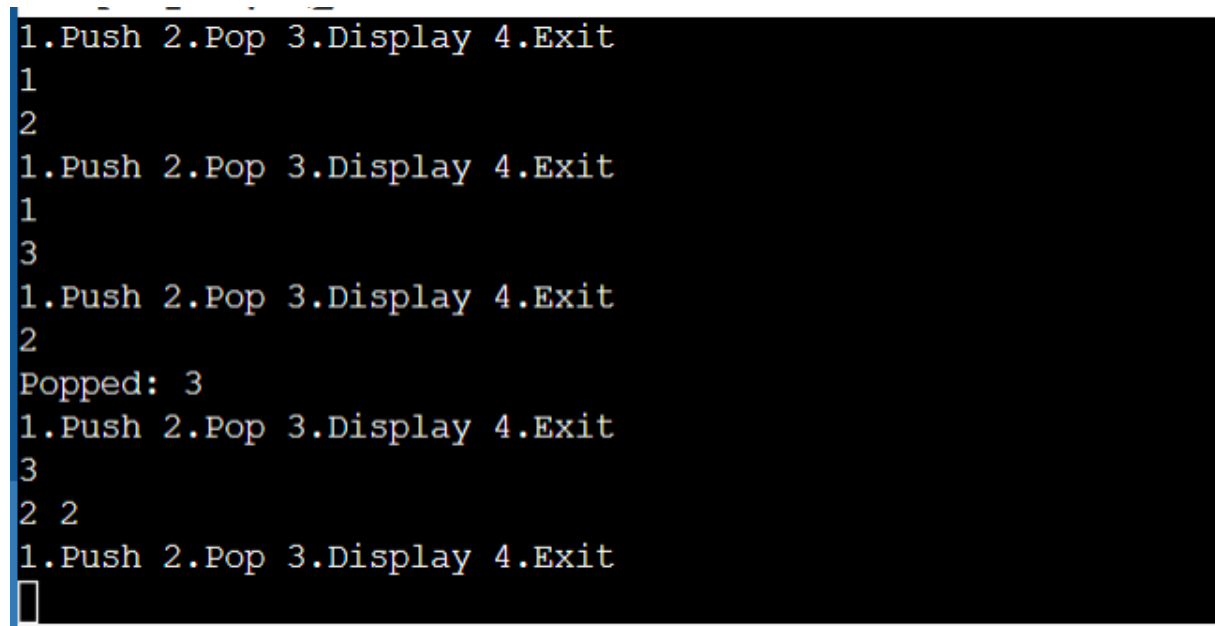
```

```

        break;
    }
} while(choice != 4);

return 0;
}

```



```

1.Push 2.Pop 3.Display 4.Exit
1
2
1.Push 2.Pop 3.Display 4.Exit
1
3
1.Push 2.Pop 3.Display 4.Exit
2
Popped: 3
1.Push 2.Pop 3.Display 4.Exit
3
2 2
1.Push 2.Pop 3.Display 4.Exit

```

2. Develop a class template for queue with enqueue and dequeue operations.

```

#include <iostream>

using namespace std;

template <class T>

class Queue {
    T arr[100];
    int front, rear;

public:
    Queue() {
        front = 0;
        rear = -1;
    }

```

```
}
```

```
void enqueue(T x) {  
    if(rear == 99) {  
        cout << "Queue Overflow\n";  
        return;  
    }  
    arr[++rear] = x;  
}
```

```
void dequeue() {  
    if(front > rear) {  
        cout << "Queue Underflow\n";  
        return;  
    }  
    cout << "Dequeued: " << arr[front++] << endl;  
}
```

```
void display() {  
    if(front > rear) {  
        cout << "Queue is empty\n";  
        return;  
    }  
    for(int i = front; i <= rear; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
};
```

```
int main() {  
    Queue<int> q;  
    int choice, val;  
  
    do {  
        cout << "1.Enqueue 2.Dequeue 3.Display 4.Exit\n";  
        cin >> choice;  
  
        switch(choice) {  
            case 1:  
                cin >> val;  
                q.enqueue(val);  
                break;  
            case 2:  
                q.dequeue();  
                break;  
            case 3:  
                q.display();  
                break;  
        }  
    } while(choice != 4);  
  
    return 0;  
}
```

```

1.Enqueue 2.Dequeue 3.Display 4.Exit
1
7
1.Enqueue 2.Dequeue 3.Display 4.Exit
1
9
1.Enqueue 2.Dequeue 3.Display 4.Exit
1
2
1.Enqueue 2.Dequeue 3.Display 4.Exit
2
Dequeued: 7
1.Enqueue 2.Dequeue 3.Display 4.Exit
3
9 2
1.Enqueue 2.Dequeue 3.Display 4.Exit

```

3. Create a class template to store and display a pair of values.

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T1, class T2>
```

```
class Pair {
```

```
    T1 a;
```

```
    T2 b;
```

```
public:
```

```
    void setValues(T1 x, T2 y) {
```

```
        a = x;
```

```
        b = y;
```

```
    }
```

```
    void display() {
```

```
        cout << "Pair: " << a << ", " << b << endl;
```

```

    }
};

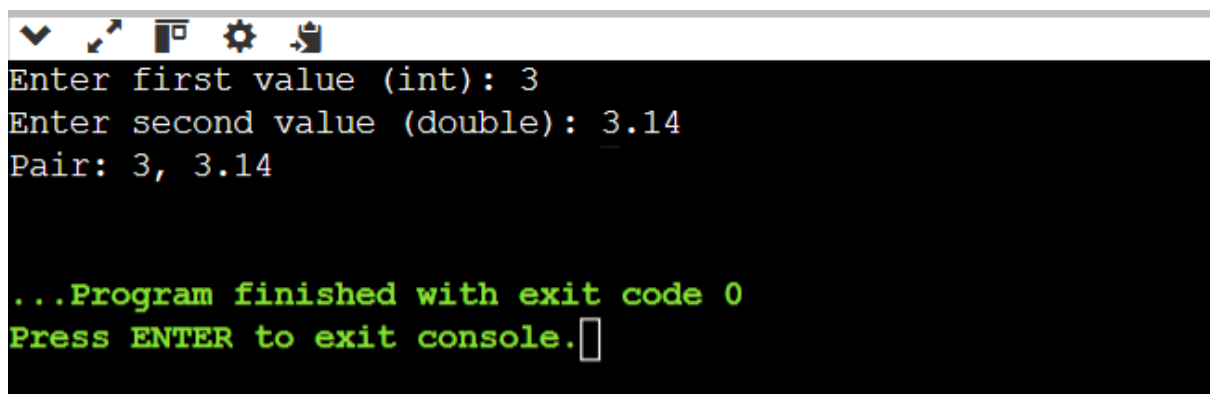
int main() {
    Pair<int, double> p;
    int x;
    double y;

    cout << "Enter first value (int): ";
    cin >> x;
    cout << "Enter second value (double): ";
    cin >> y;

    p.setValues(x, y);
    p.display();

    return 0;
}

```



```

Enter first value (int): 3
Enter second value (double): 3.14
Pair: 3, 3.14

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Create a class template for basic arithmetic operations.

```

#include <iostream>

using namespace std;

template <class T>

```

```
class Arithmetic {  
    T a, b;  
  
public:  
    void setValues(T x, T y) {  
        a = x;  
        b = y;  
    }  
  
    void add() {  
        cout << "Addition: " << a + b << endl;  
    }  
  
    void subtract() {  
        cout << "Subtraction: " << a - b << endl;  
    }  
  
    void multiply() {  
        cout << "Multiplication: " << a * b << endl;  
    }  
  
    void divide() {  
        if(b == 0) {  
            cout << "Division by zero not allowed\n";  
            return;  
        }  
        cout << "Division: " << a / b << endl;  
    }  
};
```

```

int main() {
    Arithmetic<double> obj;

    double x, y;

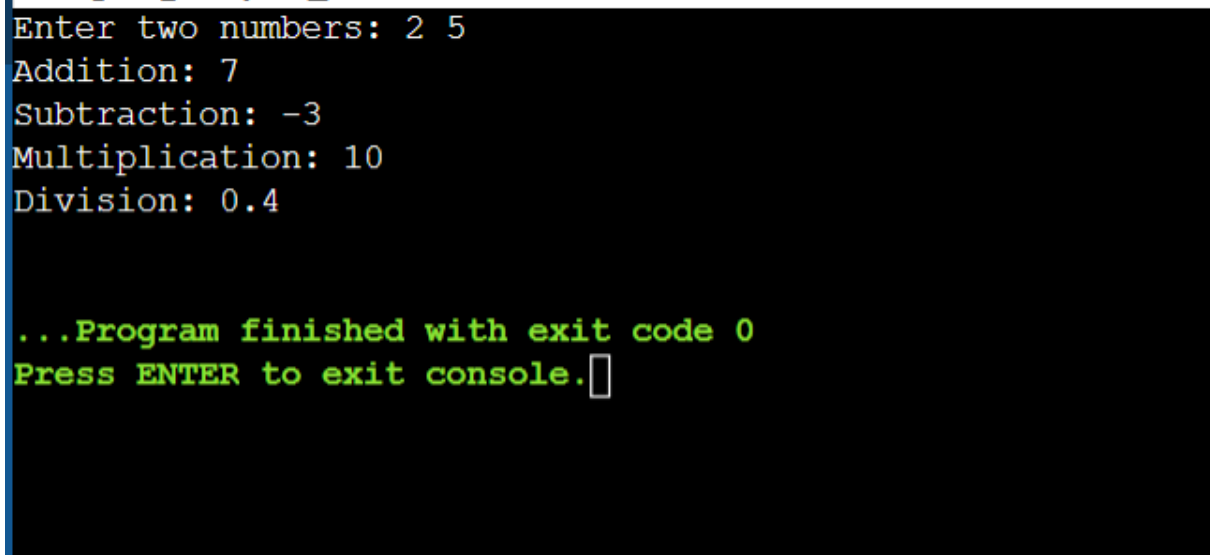
    cout << "Enter two numbers: ";
    cin >> x >> y;

    obj.setValues(x, y);

    obj.add();
    obj.subtract();
    obj.multiply();
    obj.divide();

    return 0;
}

```



The screenshot shows a terminal window with a black background. The program prompts the user to enter two numbers, and the user enters '2 5'. The program then displays the results of four arithmetic operations: Addition (7), Subtraction (-3), Multiplication (10), and Division (0.4). At the bottom, a green message indicates the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

```

Enter two numbers: 2 5
Addition: 7
Subtraction: -3
Multiplication: 10
Division: 0.4

...Program finished with exit code 0
Press ENTER to exit console.

```

5. Create a class template to input and display array elements.

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T>
```

```
class Array {
```

```
    T arr[100];
```

```
    int n;
```

```
public:
```

```
    void input() {
```

```
        cin >> n;
```

```
        for(int i = 0; i < n; i++) {
```

```
            cin >> arr[i];
```

```
        }
```

```
    }
```

```
    void display() {
```

```
        for(int i = 0; i < n; i++) {
```

```
            cout << arr[i] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
};
```

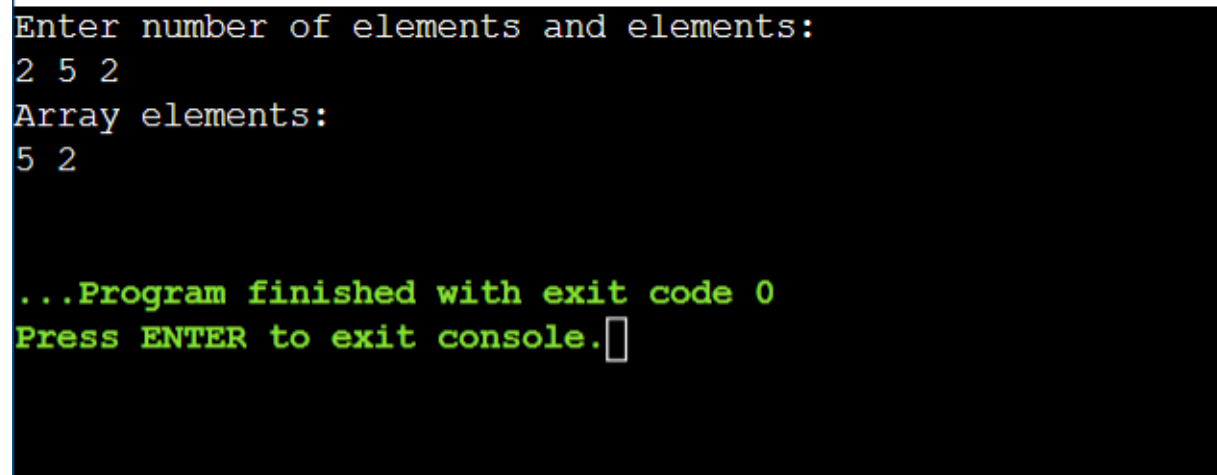
```
int main() {
```

```
    Array<int> obj;
```

```
    cout << "Enter number of elements and elements:\n";
```

```
    obj.input();
```

```
cout << "Array elements:\n";  
obj.display();  
  
return 0;  
}
```



A screenshot of a terminal window with a black background and white and green text. The text shows the program's execution flow: a prompt to enter elements, user input, array display, and a final completion message.

```
Enter number of elements and elements:  
2 5 2  
Array elements:  
5 2  
  
...Program finished with exit code 0  
Press ENTER to exit console. 
```